

Busca e Ordenação

Projeto 2 - 35 Pontos

Professor: Tadeu Zubaran

1 Introdução

Neste projeto, teremos a oportunidade de aplicar os conceitos de busca e ordenação que estudamos em sala de aula em um cenário que simula os desafios que vocês encontrarão no futuro.

Criaremos implementações usando algoritmos de busca e ordenação para trabalhar com textos. Durante esse processo, teremos aulas práticas no laboratório.

2 Definição

Os alunos tem os seguintes desafios de implementação:

1. **KMP:** Encontrar ocorrências de palavras em um texto. O usuário deve digitar uma palavra, e o algoritmo encontra uma a uma, em ordem, as ocorrências destas palavras no texto. Esse algoritmo deve ser implementado com o Knuth-Morris-Pratt (KMP)
2. **Uso de wildscore:** Permitir o uso de um símbolo coringa *, no item anterior, que significa qualquer outro símbolo.
3. **Buscas de várias palavras:** Implementar busca de várias palavras em paralelo usando o algoritmo Aho-Corasick. E em seguida fazer uma análise de desempenho comparativo da técnica ingênua de buscas em texto, KMP e Aho-Corasick
4. **Cifra e Quebra da cifra:** Implementar uma ferramenta que permita ao usuário criptografar e descriptografar mensagens usando uma bijeção arbitrária entre os símbolos do alfabeto. Nesta técnica cada letra é mapeada para qualquer outra, onde esse mapeamento é dado pela chave. Note que cada letra diferente deve ser mapeada para uma letra distinta para que a

decriptografia seja possível. Em outras palavras a chave é uma permutação das letras do alfabeto. Esta criptografia permite $n!$ chaves distintas, onde n é o número de símbolos da alfabeto. Posteriormente vocês devem pesquisar e implementar técnicas para quebrar a criptografia do item anterior. Esse código deve receber um texto (grande) já criptografado, mas não a chave (mapeamento das letras). Você pode escolher trabalhar com textos em português ou inglês. Após análise o algoritmo deve ser capaz de mostrar o texto original (antes de ter sido criptografado) e a chave.

5. **Marcação de erros ortográficos:** Encontrar palavras em um texto que não estão em um dicionário. Caso o algoritmo não seja eficiente o suficiente você pode se limitar a palavras que começam com "a" somente. Alternativamente você pode se limitar a palavras que iniciam com "q" somente
6. **Sugestões de correção:** Sugerir correções das palavras encontradas no item anterior, utilizando a distância de string da palavra errada com palavras do dicionário. Novamente você pode fazer as mesmas limitações do item anterior, caso necessário.
7. **Data mining:** Desenvolver uma aplicação que analise documentos de texto em busca de informações específicas, como datas, números de telefone ou endereços de e-mail.
8. **Classificação de textos:** Criar um sistema que classifica documentos de texto em categorias específicas com base em seu conteúdo. Isso pode ser aplicado a uma variedade de tarefas, como classificar notícias em categorias ou analisar sentimentos em análises de produtos.
9. **Visualização:** Desenvolver um gerador de nuvem de palavras que visualize as palavras mais frequentes em um conjunto de textos. Isso pode ser usado para análise textual e resumo de conteúdo.
10. **Compressão e descompressão de textos:** Pesquisar em implementar técnicas de compressão e descompressão de textos (p.ex. código de Huffman)

Os alunos devem escolher algumas destas tarefas para executar. A **tarefa 1 é obrigatória** para todos os alunos. Sua avaliação levará em conta a qualidade e quantidade dos desafios escolhidos, assim como a qualidade das implementações. **Não é esperado dos alunos que todas as tarefas sejam executadas.**

3 Entrega

A entrega consiste de 33 coisas.

- Upload do código,e apresentação no google class.
- Apresentação de seminário para os colegas.

- pequeno relatório explicando o quê foi feito.

Trabalhos sem qualquer uma das 2 componentes não serão considerados.

Na apresentação os alunos devem mostrar o código funcionando. Ou executando no momento, ou com vídeo mostrando execução.

4 Avaliação

Critérios de avaliação:

- Desafio da seleção de tarefas.
- Qualidade da implementação e projeto.
- Implementação de algoritmos associados a busca e ordenação. Não use implementações prontas na linguagem para este tipo de algoritmo.
- Clareza e corretude do código e da sua explicação.
- Qualidade e explicação das decisões de projeto.
- Código implementado com boas práticas de programação.
- Qualidade de apresentação para os colegas e participação com perguntas nas apresentações dos outros grupos

Não copie o código. Faça seu próprio código! Plágio receberá nota 0.